

# **COMSATS UNIVERSITY ISLAMABAD**

## **LAHORE CAMPUS**



---

### **CSC241 Theory**

### **ASSIGNMENT:1**

### **Object-Oriented Analysis of a Smart Campus**

### **Allocation System**

**Submitted by:** Aamna Basit

**Reg no:** SP26-BAI-001

**Submitted to:** Dr. Muhammad Shahid Bhatti

**Section:** BAI

**Selected Condition:** Privacy First (Last Digit: 1)

**Date:** 24/09/26

# Task 1: Stakeholder and Requirement Analysis

## 1. Key Stakeholders and Their Needs

1. **Student:** Needs a simple process to reserve spaces for self-study or group project meetings without facing unexpected booking delays.
2. **Faculty Member:** Requires guaranteed booking for teaching, makeup lectures and official academic event.
3. **Maintenance Staff:** Needs immediate ability to lock and mark a room restricted whenever maintenance issues occur.
4. **Security Staff:** Requires access to traceable records showing who occupied a space and when, ensuring safety on campus.

## 2. Requirement Conflicts and Resolutions

- **Conflict 1: Privacy First vs. Security Audit Trail**
  - **Problem:** Security needs detailed records of every person, whereas regular users expect privacy regarding who is using a space and why.
  - **Decision: Role-Based Visibility.** The system will record complete entry details in a secure database, but ordinary users will only see whether a room is Available or Booked.
- **Conflict 2: Faculty Priority vs. Student Fair Access**
  - **Problem:** When professors can cancel room bookings at any time, students suffer unexpected disruptions.
  - **Decision: Notice Buffer & Auto-Relocation.**
  - Teachers must give 24 hours' notice to take a room. If you lose your room, the system will automatically find you a new one.
- **Conflict 3: High Room Utilization vs. Maintenance Blocking**
  - **Problem:** Safety comes first, so unsafe rooms are locked immediately and all bookings are canceled.
  - **Decision: Immediate Safety Lock.** Maintenance safety holds absolute priority over room occupancy targets. If a room is flagged as unsafe, all current and upcoming bookings for that space are instantly put on hold.

## 3. Facts Given vs. Assumptions Introduced

| Facts                                                                                                 | Assumptions                                                                                                        |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Users include students, faculty, administrators, security, maintenance, and disability support staff. | Public room searches will strictly display time slots as BOOKED, hiding individual names ( <b>Privacy First</b> ). |

|                                                                                |                                                                                                |
|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| A single individual can hold more than one role.                               | Unclaimed bookings are automatically canceled if the user fails to check in within 15 minutes. |
| Maintenance administration can block unsafe rooms immediately.                 | Students can book spaces a maximum of 14 days in advance to prevent room hoarding.             |
| Emergency situations can alter, relocate, or cancel existing room reservations | Faculty overrides require at least a 24-hour advance notice buffer.                            |

## Task 2: Object Oriented Conceptual Model

### 1. Proposed Classes and Responsibilities

#### 1. User

- *Responsibility:* Stores account details and identifies a person using the system.
- *Attributes:* `userId: String`, `name: String`, `email: String`
- *Operations:* `getName()`, `getRoles()`

#### 2. Role

- *Responsibility:* Defines specific permissions and priority access associated with a user role.
- *Attributes:* `roleName: String`, `priorityLevel: int`
- *Operations:* `canOverride()`, `getPermissions()`

#### 3. Space

- *Responsibility:* Represents a physical room on campus and manages its current operational state.
- *Attributes:* `roomId: String`, `capacity: int`, `isUnsafe: boolean`
- *Operations:* `checkAvailability()`, `setUnsafeBlock()`

#### 4. Reservation

- *Responsibility:* Tracks an individual booking request, connecting a user, a space, and a specific time slot.
- *Attributes:* `reservationId: String`, `status: String`, `purpose: String`
- *Operations:* `confirmBooking()`, `cancelBooking()`

#### 5. TimeSlot

- *Responsibility:* Handles the start and end times for reservations and checks for timing overlaps.
- *Attributes:* `startTime: String`, `endTime: String`
- *Operations:* `checkOverlap()`, `getDuration()`

#### 6. AllocationPolicy

- *Responsibility:* Evaluates booking rules, checks user quotas, and resolves scheduling conflicts.
- *Attributes:* `maxHoursPerWeek: int`, `leadTimeHours: int`

- *Operations:* validateBooking(), resolveConflict()

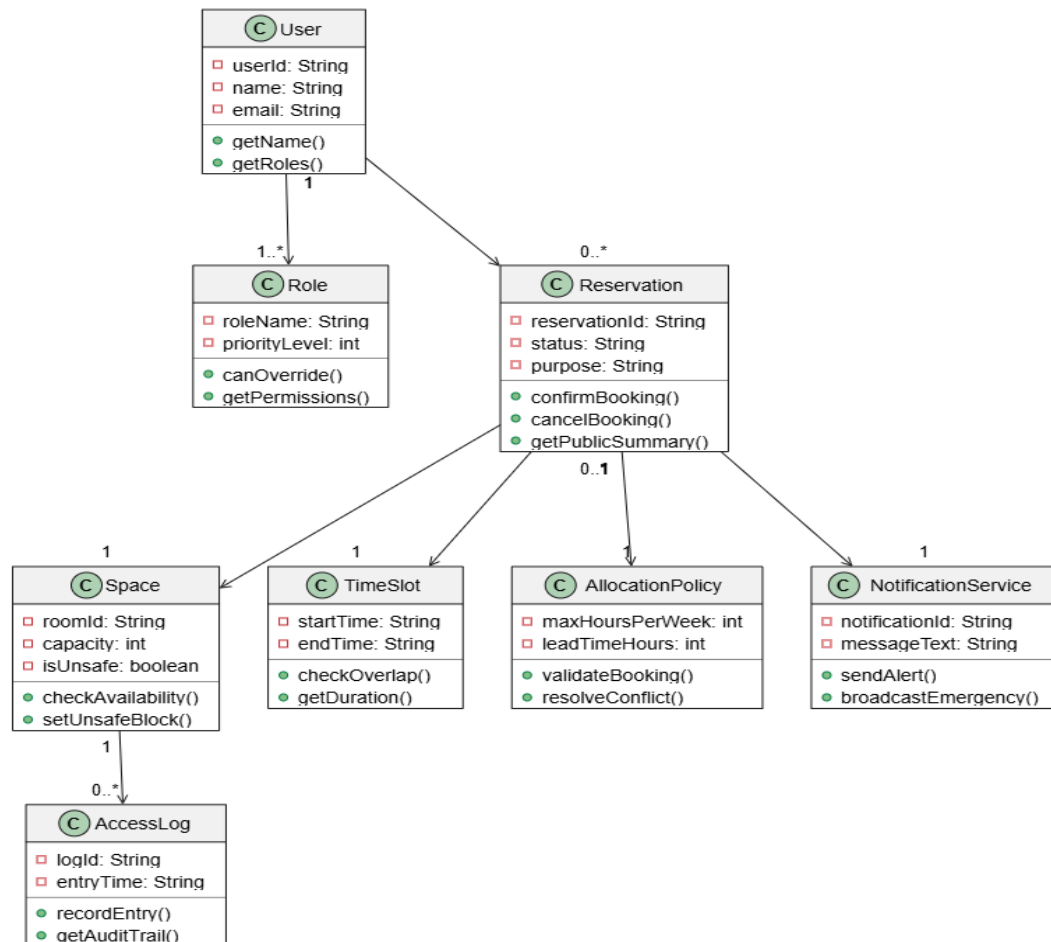
## 7. AccessLog

- *Responsibility:* Maintains a secure log of room entries and booking actions for security auditing.
- *Attributes:* logId: String, entryTime: String
- *Operations:* recordEntry(), getAuditTrail()

## 8. NotificationService

- *Responsibility:* Sends automated alerts to users regarding booking updates, cancellations, or emergencies.
- *Attributes:* notificationId: String, messageText: String
- *Operations:* sendAlert(), broadcastEmergency()

## 2. Conceptual Class Diagram



## 3. Responsibility Separation (Avoiding a "God Class")

To avoid building a single massive controller class that tries to manage everything at once, my design distributes responsibilities across smaller classes:

- **Space** focuses strictly on physical room capacity and safety conditions.

- **AllocationPolicy** independently handles complex business rules and scheduling decisions.
- **Reservation** tracks individual booking lifecycle states.
- **NotificationService** focuses solely on messaging users.

This modular structure ensures that modifying one part of the system does not break unrelated components.

## 2. Encapsulation (Hiding Internal State)

- **Space:**
  - *Hidden Data:* `isUnsafe` (kept private so external classes cannot alter room safety directly).
  - *Controlled Operation:* `setUnsafeBlock(boolean status)` allows maintenance staff to safely update room availability while automatically initiating reservation cancellations.
- **Reservation:**
  - *Hidden Data:* purpose and user details (kept private to uphold the **Privacy First** requirement).
  - *Controlled Operation:* `getPublicSummary()` returns only "BOOKED" or "AVAILABLE", making sure sensitive user details remain hidden from public view.
- **User:**
  - *Hidden Data:* roles list (kept private).
  - *Controlled Operation:* `addRole(Role newRole)` ensures new permissions are added through proper checks instead of being modified directly.

## 3. Four System Invariants (Always True Rules)

1. **Invariant 1:** A `TimeSlot` must always have an end time that occurs after its start time (`startTime < endTime`).
2. **Invariant 2:** A `Space` cannot have two active, confirmed reservations during the exact same `TimeSlot`.
3. **Invariant 3:** If `Space.isUnsafe == true`, the system must prevent any new reservations from being created for that room.
4. **Invariant 4 (Privacy First):** Public booking queries must never display the identity or reason for booking of the user holding a reservation.

### ***Role of Encapsulation:***

Keeping variables private and routing all changes through methods allows us to place if validation checks inside those methods, ensuring no invalid data can ever corrupt the system's state.

## Task 4: Designs and Decision

### 1. Comparison: Inheritance vs. Composition (Role Objects)

- **Option A (Inheritance - extends):** Creating subclasses from `User` (e.g., `Student` extends `User`, `Faculty` extends `User`).

- **Option B (Composition / Role Objects - has-a):** Giving User a list of roles (User has-a List<Role>).

| Evaluation criteria        | Option A:<br><br>Inheritance (extends)                                                                                             | Option B:<br><br>Composition (has-a)                                                                 |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Handling Multiple Roles    | <b>Difficult:</b> Java does not support multiple class inheritance. Modeling a Teaching Assistant (Student + Staff) becomes messy. | <b>Flexible:</b> A single User object can easily store multiple Role instances inside a simple list. |
| Changing Roles Dynamically | <b>Rigid:</b> An object cannot change its class type at runtime during a semester.                                                 | <b>Easy:</b> Roles can be added or removed from a user's list at any time using simple methods.      |
| Code Maintenance           | <b>High Coupling:</b> Adding new campus roles creates a bloated class tree that is hard to manage.                                 | <b>Low Coupling:</b> New roles can be created as new objects without modifying the core User class.  |

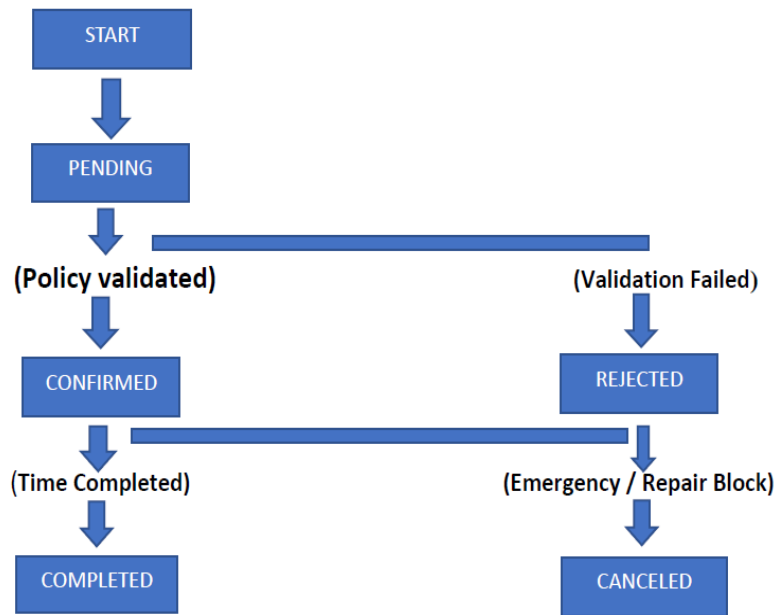
## 2. Final Decision and Justification

**Selected Design: Option B (Composition / Role Objects).**

- *Justification:* The scenario states that an individual may hold multiple roles simultaneously and permissions can change over the semester. Instead of creating a giant, complex family tree of classes, Java composition lets you easily change an object's role while the program runs.

## Task 5: Behaviour and Edge Cases

### 1. Reservation State Transitions



## 2. Edge Case Handling Logic

### 1. Two Simultaneous Requests for the Same Space:

- *Resolution:* The AllocationPolicy evaluates both requests. The request with the higher Role.priorityLevel (e.g., Faculty over Student) receives the reservation. If priority levels are identical, a First-Come-First-Served rule based on submission timestamps breaks the tie.

### 2. Maintenance Block Placed After Confirmation:

- *Resolution:* Maintenance flags the room using Space.setUnsafeBlock(true). The Space object triggers an event to NotificationService, which automatically cancels active bookings and notifies affected users.

### 3. Emergency Cancellation or Relocation:

- *Resolution:* An administrator initiates an emergency state. The system logs the incident in AccessLog for security audits, cancels existing bookings, and directs AllocationPolicy to search for available alternative rooms of matching capacity.

## 4. Responsibility Assignment

| Action / Decision            | Assigned Class   | Justification                                                  |
|------------------------------|------------------|----------------------------------------------------------------|
| Priority Conflict Resolution | AllocationPolicy | Keeps business rules separate from basic data classes.         |
| Locking Unsafe Rooms         | Space            | Directly owns physical room availability data                  |
| Enforcing Data Privacy       | Reservation      | Controls public visibility and hides sensitive booking fields. |

## Task 6: Independent Exploration (Role-Based Access Control)

### Concept Definition & Model Application

**Role-Based Access Control (RBAC)** is a security management approach where system permissions are tied to specific roles rather than directly to individual user accounts. Instead of assigning permissions directly to a person like "Aamna", rights are assigned to roles such as "Student", "Faculty", or "Security Staff", and users are then assigned to those roles.

### How it Improves Our Smart Campus System?

RBAC directly solves our **Privacy First** requirement in the room booking system. When a user searches for a room, the system checks their role before showing any information:

- If the user has an **Ordinary User** role (like a regular Student), the system only shows if the room is Available or Booked. It hides the name of the person who booked it and the purpose of the meeting.
- If the user has a **Security** or **Admin** role, the system opens full access so they can see exact student names and details for safety and emergency audits.

This keeps student data safe without breaking the system for campus security.

### References

1. **Ferraiolo, D. F., & Kuhn, D. R. (1992).** *Role-based access controls*. 15th National Computer Security Conference, pp. 554–563.
2. **Sandhu, R. S., et al. (1996).** *Role-based access control models*. IEEE Computer, 29(2), pp. 38–47.